Dokumentacja techniczna modułu generatory

17 października 2025

Spis treści

1 Główne komponenty **3**

2 Klasa Seed **3**
2.1 Cel i opis . . . 3
2.2 Struktura klasy . . . 3
2.3 Szczegółowe omówienie elementów . . . 4
2.4 Zasada działania . . . 4
2.5 Algorytm genetyczny . . . 4

3 Interfejs `IRandomSource` **5**
3.1 Opis . . . 5
3.2 Szczegółowe omówienie elementów . . . 5

4 Interfejs `IGenerator<T>` **5**
4.1 Opis . . . 5
4.2 Szczegółowe omówienie elementów . . . 5

5 Generatory skalarne i wektorowe **6**
5.1 FloatGenerator . . . 6
5.2 Szczegółowe omówienie elementów . . . 6
5.3 Tryby generacji . . . 6
5.4 IntGenerator . . . 6
5.5 Szczegółowe omówienie elementów . . . 7

6 Generatory wektorowe **7**
6.1 Interfejs generyczny . . . 7
6.2 Opis . . . 7
6.3 Podstawowa implementacja . . . 7
6.4 Opis działania . . . 8

7 Interfejsy i implementacje generatorów wzorców ataku **8**
7.0.1 Interfejs główny: `IAttackPatternGenerator` . . . 8
7.0.2 Interfejs układu wzorca: `IAttackPatternLayout` . . . 8
7.0.3 Klasa abstrakcyjna pomocnicza: `AbstractAttackPatternGenerator` 9
7.0.4 Przykładowy generator: `BurstAttackPatternGenerator` . . . 10
7.0.5 Przykładowy układ: `BurstLayout` . . . 11

8 Generator szumu **11**

1 Główne komponenty

- **Seed** — źródło deterministycznej entropii, podstawowy element kontroli losowości.
- **IRandomSource** — niskopoziomowy interfejs generujący liczby losowe.
- **IGenerator<T>** — generyczny interfejs dla wszystkich typów generatorów.
- **Generatory skalarne i wektorowe** — `FloatGenerator`, `IntGenerator`, `Vector2Generator`, `Vector3Generator`.
- **Attack Pattern Generators** — moduły generujące złożone, losowe wzorce zachowań przeciwników.

2 Klasa Seed

2.1 Cel i opis

Klasa `GlobalSeed` jest deterministycznym źródłem entropii. Każdy generator wykorzystuje tworzony w niej `Seed` do stworzenia niezależnego strumienia losowych wartości, co gwarantuje powtarzalność w wynikach generacji.

2.2 Struktura klasy

```
public class Seed
{
    public ulong Value { get; private set; }
    public int Id { get; private set; }
    public string ParentName { get; private set; }

    public Seed(string key, string parentName = null)
    { ... }
}

[Serializable]
public class GlobalSeed
{
    public ulong Base { get; private set; }
    public string Name { get; private set; }

    private List<Seed> subSeeds;

    public GlobalSeed(ulong baseSeed, string name = null);

    public Seed GetSubSeed(string key);
    public Seed NextSubSeed(string key);

    public string ToShortHash();
    public string Serialize();
    public static Seed FromString(string serialized);
```

```
}
```

2.3 Szczegółowe omówienie elementów

- `Base` — liczba całkowita będąca bazą entropii; często inicjalizowana z `System.Random` lub hash'a identyfikatora sesji.
- `Name` i `Description` — metadane ułatwiające debugowanie i analizę.
- `subSeeds` — mapa podrzędnych seedów dla niezależnych kontekstów (np. „EnemyWaveGen", „LootDropGen").

Funkcje:

- `GetSubSeed(key)` — zwraca liczbę całkowitą będącą deterministycznym pochodnym seeda.
- `NextSubSeed(key)` — tworzy nowy obiekt klasy `Seed`. Jest to wartość operacji bitowej na `GlobalSeed` oraz kluczu.
- `ToShortHash()` — przydatne do logowania krótkich identyfikatorów losowości.
- `Serialize()` — konwersja do postaci tekstowej (np. do zapisania w save).

2.4 Zasada działania

`GlobalSeed` generuje sub-seedy deterministycznie na podstawie klucza tekstowego:

$$SubSeed = Mix(Base \oplus Hash(Key))$$

Dzięki temu różne systemy w grze (np. generacja fal przeciwników, lootu, trajektorii pocisków) mogą korzystać z niezależnych strumieni RNG bez interferencji.

2.5 Algorytm genetyczny

Zgodnie z założeniami metody konstruujące seedy (a w dalszej mierze także metody umożliwiające modyfikację parametrów innych klas) powinny umożliwiać wykorzystanie do tego obiektów tej samej klasy, które przekazywałyby swoje wartości w sposób zbliżony do algorytmu genetycznego. W klasycznym algorytmie genetycznym rodzice są wybierani z populacji na podstawie tzw. funkcji dopasowania (fitness). Tutaj rodzicami są po prostu dwa istniejące seedy, między którymi wystąpi krzyżowanie (crossover), czyli wymieszanie wartości bajtów obu rodziców w taki sposób, by dana wartość przekazana dziecku miała 50% szans na pochodzenie od jednego z rodziców (teoretycznie możliwy jest dobór większej liczby rodziców niż 2). Po krzyżowaniu dziecko przechodzi etap mutacji. To sposób na wprowadzenie nowej informacji do populacji i zapobieganie stagnacji. Każdy bajt ma określone prawdopodobieństwo, np. 5%, że zostanie zmodyfikowany. Modyfikacja opierałaby się na działaniach bitowych i mogłaby polegać m.in. na: wykonywaniu bit flipów, nadpisywaniu ciągu bitów określonymi wartościami, czy xorowaniu z określonymi maskami bitowymi. Końcowy etap algorytmu stanowiłaby walidacja, czyli sprawdzenie czy wynik nie jest zbyt podobny do któregoś z rodziców, np. obliczając Hamming Distance.

3 Interfejs IRandomSource

3.1 Opis

Interfejs niskiego poziomu odpowiedzialny za generowanie surowych wartości losowych. Pozwala na tworzenie własnych implementacji generatorów.

```
public interface IRandomSource
{
    ulong NextUInt64();
    double NextFloat();
    IRandomSource Clone();
}
```

3.2 Szczegółowe omówienie elementów

- `NextUInt64()` — zwraca kolejną 64-bitową wartość losową.
- `NextFloat()` — konwertuje wynik na wartość zmiennoprzecinkową z zakresu [0,1).
- `Clone()` — tworzy kopię stanu RNG; pozwala na symulacje „podglądowe”, np. do generacji `Preview()`.

Interfejs nie zależy od żadnej z klas Unity, co pozwala na jego testowanie i użycie w kontekście edytora lub serwera.

4 Interfejs IGenerator<T>

4.1 Opis

Podstawowy interfejs dla wszystkich generatorów wartości losowych. Zdefiniowany generycznie, aby wspierać zarówno typy proste, jak i złożone obiekty.

```
public interface IGenerator<T>
{
    T Generate(IRandomSource source);
    T Generate(Seed seed, string context = null);
    T Preview(ulong pseudo);
}
```

4.2 Szczegółowe omówienie elementów

- `Generate(IRandomSource source)` — podstawowy sposób generacji, wykorzystywany przez inne klasy biblioteki.
- `Generate(Seed seed, string context)` — wygodna metoda dla twórcy gier; konwertuje `Seed` na `IRandomSource` i dodaje kontekst.
- `Preview(ulong pseudo)` — generuje symulowaną wartość bez modyfikacji stanu RNG (np. do wyświetlenia w edytorze Unity).

5 Generatory skalarne i wektorowe

5.1 FloatGenerator

```
[Serializable]
public class FloatGenerator : IGenerator<float>
{
    public enum Mode { Uniform, Normal, Triangular, Step, Curve }
    public Mode mode;
    public float min, max;
    public float mean, stdDev;
    public int steps;
    public AnimationCurve curve;

    public float Generate(IRandomSource source) { ... }
}
```

5.2 Szczegółowe omówienie elementów

- `mode` — tryb rozkładu (jednolity, normalny, trójkątny itp.).
- `min`, `max` — zakres wartości w trybie `Uniform` lub `Triangular`.
- `mean`, `stdDev` — parametry rozkładu normalnego.
- `steps` — liczba kroków w trybie schodkowym (`Step`).
- `curve` — niestandardowy rozkład określony przez użytkownika w Unity.

5.3 Tryby generacji

- **Uniform** — równomierny rozkład w zakresie $[min, max]$.
- **Normal** — rozkład Gaussa o parametrach `mean`, `stdDev`.
- **Triangular** — losowanie wartości z przewagą w środku przedziału.
- **Curve** — losowanie, a następnie remapowanie przez `AnimationCurve`.

5.4 IntGenerator

```
[Serializable]
public class IntGenerator : IGenerator<int>
{
    public int min;
    public int max;

    public int Generate(IRandomSource source) { ... }
}
```

5.5 Szczegółowe omówienie elementów

- `min`, `max` — granice przedziału.
- Generacja opiera się na `NextUInt64()` i redukcji modulo, aby zachować równomierność.

6 Generatory wektorowe

6.1 Interfejs generyczny

```
public interface IVectorGenerator<TScalar> : IGenerator<List<
   TScalar>>
{
    int Dimension { get; }
    IGenerator<TScalar> ScalarGenerator { get; }
}
```

6.2 Opis

- `IVectorGenerator` definiuje kontrakt dla wszystkich generatorów wektorowych,
- każdy wektor jest reprezentowany jako `List<TScalar>` — np. `List<float>` lub `List<int>`,
- liczba wymiarów jest określana przez pole `Dimension`.

6.3 Podstawowa implementacja

```
[Serializable]
public class VectorGenerator<TScalar> : IVectorGenerator<TScalar>
{
    public int Dimension { get; private set; }
    public IGenerator<TScalar> ScalarGenerator { get; private set;
       }

    public VectorGenerator(IGenerator<TScalar> scalarGenerator, int
        dimension = 2)
    { ... }

    public List<TScalar> Generate(IRandomSource source)
    { ... }

    public List<TScalar> Generate(Seed seed, string context = null)
    { ... }

    public List<TScalar> Preview(ulong pseudo)
    { ... }
}
```

6.4 Opis działania

- Klasa `VectorGenerator` generuje wektor dowolnego wymiaru, tworząc listę wartości liczbowych.
- Każda składowa jest generowana przy użyciu tego samego `ScalarGenerator`.

7 Interfejsy i implementacje generatorów wzorców ataku

7.0.1 Interfejs główny: `IAttackPatternGenerator`

```
public interface IAttackPatternGenerator : IGenerator<AttackPattern
    >
{
    FloatGenerator DurationGenerator { get; set; }
    IntGenerator EventCountGenerator { get; set; }

    void Validate();
}
```

Opis:

- Interfejs definiujący kontrakt dla wszystkich generatorów wzorców ataku.
- Zapewnia dostęp do podstawowych generatorów:
  - `DurationGenerator` — odpowiada za losowanie długości całego wzorca ataku.
  - `EventCountGenerator` — określa liczbę pojedynczych wydarzeń (`AttackEvent`) w danym wzorcu.
- Dziedziczy po `IGenerator<AttackPattern>`, dzięki czemu każdy generator wzorca integruje się z systemem seeda i źródła losowości.
- Metoda `Validate()` służy do walidacji konfiguracji generatora przed jego użyciem (np. sprawdza, czy wszystkie pola zostały przypisane).

7.0.2 Interfejs układu wzorca: `IAttackPatternLayout`

```
public interface IAttackPatternLayout
{
    List<AttackEvent> GenerateEvents(IRandomSource source, int
        eventCount, float duration);
}
```

Opis:

- Interfejs opisujący sposób rozmieszczenia poszczególnych wydarzeń ataku w czasie i przestrzeni.
- Każdy układ definiuje, jak mają wyglądać kolejne `AttackEvent` obiektu `AttackPattern`.
- Argumenty metody:
  - `source` — źródło losowości wykorzystywane do generowania parametrów.
  - `eventCount` — liczba wygenerowanych wydarzeń.
  - `duration` — całkowity czas trwania wzorca.
- Umożliwia tworzenie różnych stylów zachowań przeciwników, np.:
  - **BurstLayout** — skupienie wszystkich ataków w krótkim czasie,
  - **WaveLayout** — równomierne rozłożenie ataków w czasie,
  - **SpiralLayout** — rotacyjne zmiany kierunków między kolejnymi atakami.

7.0.3 Klasa abstrakcyjna pomocnicza: `AbstractAttackPatternGenerator`

```
[Serializable]
public abstract class AbstractAttackPatternGenerator :
   IAttackPatternGenerator
{
    public FloatGenerator DurationGenerator { get; set; }
    public IntGenerator EventCountGenerator { get; set; }

    public abstract AttackPattern Generate(IRandomSource source);
    public virtual AttackPattern Generate(Seed seed, string context
        = null);
    public virtual AttackPattern Preview(ulong pseudo);
    public virtual void Validate();
}
```

Opis:

- Klasa abstrakcyjna zapewniająca podstawową strukturę dla generatorów wzorców ataku.
- Zawiera definicje najważniejszych metod generujących wzorce oraz właściwości wymaganych przez interfejs.
- `DurationGenerator` i `EventCountGenerator` stanowią minimalny zestaw komponentów wymaganych do stworzenia wzorca.
- Metody:
  - `Generate(IRandomSource source)` — główna metoda generująca obiekt `AttackPattern` z wykorzystaniem źródła losowości.

- `Generate(Seed seed, string context)` — alternatywna metoda deterministyczna, pozwalająca na generowanie na podstawie seeda i kontekstu (np. nazwy fali lub poziomu).
- `Preview(ulong pseudo)` — metoda służąca do symulacji (podglądu) wzorca bez modyfikowania rzeczywistego stanu generatora losowości.
- `Validate()` — metoda sprawdzająca poprawność konfiguracji przed generowaniem (np. czy nie brakuje wymaganych generatorów).

- Sama klasa nie implementuje logiki generowania — pozostawia ją klasom pochodnym.

7.0.4 Przykładowy generator: `BurstAttackPatternGenerator`

```
[Serializable]
public class BurstAttackPatternGenerator :
   AbstractAttackPatternGenerator
{
    public FloatGenerator TimeOffsetGenerator { get; set; }
    public VectorGenerator<float> DirectionGenerator { get; set; }
    public FloatGenerator SpeedGenerator { get; set; }
    public IntGenerator DamageGenerator { get; set; }
    public FloatGenerator SpreadAngleGenerator { get; set; }

    public IAttackPatternLayout Layout { get; set; }

    public override AttackPattern Generate(IRandomSource source);
}
```

Opis:

- Implementacja konkretnego generatora wzorca ataku typu **Burst** — krótkiej serii intensywnych ataków.
- Rozszerza `AbstractAttackPatternGenerator`, dzięki czemu integruje się z systemem seedów i losowości.
- Atrybuty:
  - `TimeOffsetGenerator` — określa przesunięcie czasowe pomiędzy kolejnymi atakami.
  - `DirectionGenerator` — losuje kierunek ataku (np. dla pocisków, wiązek lub obszarowych efektów).
  - `SpeedGenerator` — generuje prędkość pocisków lub efektów.
  - `DamageGenerator` — odpowiada za siłę pojedynczego ataku.
  - `SpreadAngleGenerator` — określa kąt rozproszenia między kolejnymi atakami.
  - `Layout` — instancja układu, który definiuje sposób rozmieszczenia wygenerowanych wydarzeń (`AttackEvent`) w czasie.

- Metoda `Generate(IRandomSource source)`:
  - tworzy obiekt `AttackPattern` z listą wydarzeń wygenerowanych przez `Layout`,
  - korzysta z wewnętrznych generatorów do przypisania parametrów poszczególnym atakom.

7.0.5 Przykładowy układ: `BurstLayout`

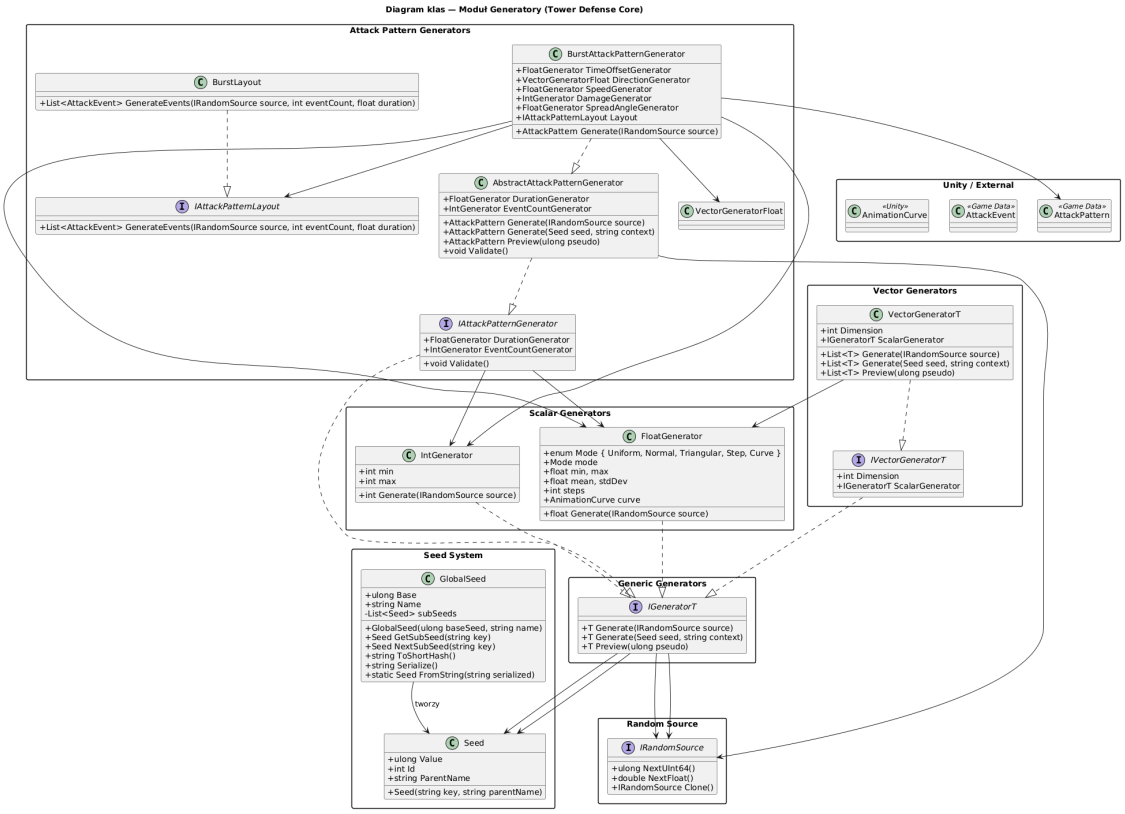
```
public class BurstLayout : IAttackPatternLayout
{
    List<AttackEvent> GenerateEvents(IRandomSource source, int
        eventCount, float duration);
}
```

Opis:

- Przykładowa implementacja prostego układu rozmieszczenia ataków.
- Typowy dla ataków o charakterze „burst” — skupionych w krótkim czasie, często na losowych kierunkach.
- Metoda `GenerateEvents(...)` tworzy listę wydarzeń (`AttackEvent`) równomiernie lub losowo rozmieszczonych w ramach trwania wzorca.
- Może być rozszerzana w przyszłości o logikę specyficzną dla danego typu wroga (np. różne rozrzuty lub zależność od prędkości ruchu celu).

8 Generator szumu

Do tego celu biblioteka importuje i wykorzystuje `FastNoise Lite`, otwartoźródłowy projekt, będący zestawem algorytmów do generacji szumu.



Rysunek 1: Diagram UML.